

ASSIGNMENT-9.1

(Documentation Generation -Automatic
documentation and code comments)

HTNO:2303A51919

BATCH NO:28

Problem 1:

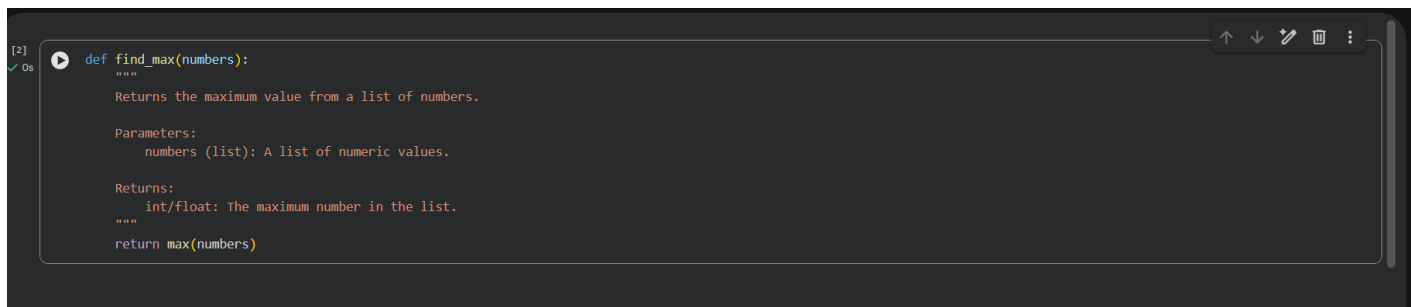
Consider the following Python function:

```
def find_max(numbers):  
    return max(numbers)
```

Task:

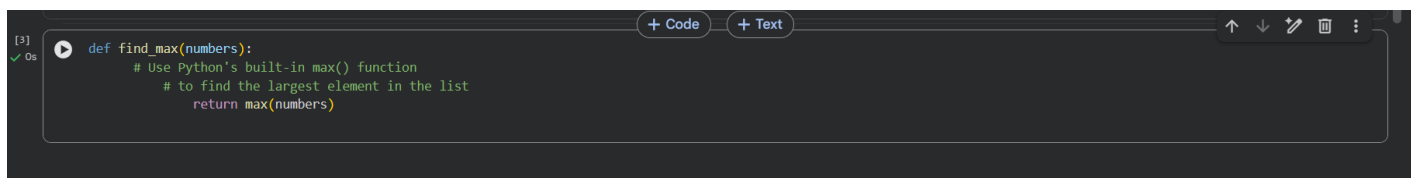
- Write documentation for the function in all three formats:
 - (a) Docstring
 - (b) Inline comments
 - (c) Google-style documentation
- Critically compare the three approaches. Discuss the advantages, disadvantages, and suitable use cases of each style.
- Recommend which documentation style is most effective for a mathematical utilities library and justify your answer.

(a)Docstring



```
[2] ✓ Os def find_max(numbers):  
    """  
    Returns the maximum value from a list of numbers.  
  
    Parameters:  
        numbers (list): A list of numeric values.  
  
    Returns:  
        int/float: The maximum number in the list.  
    """  
    return max(numbers)
```

(b) Inline Comments



```
[3] ✓ Os def find_max(numbers):  
    # Use Python's built-in max() function  
    # to find the largest element in the list  
    return max(numbers)
```

(c) Google-Style Documentation

```

7] def find_max(numbers):
  """
  Finds the maximum value in a list of numbers.

  Args:
    numbers (list): A list containing numeric values.

  Returns:
    int or float: The largest value in the list.
  """
  return max(numbers)

```

Comparison of Documentation Styles:

Style	Advantages	Disadvantages	Use Cases
Docstring	Readable, supported by tools	Less structured	Small projects
Inline Comments	Easy to understand logic	No API info	Explaining logic
Google Style	Structured, professional	Slightly lengthy	Libraries, APIs

Recommendation:

Google-style documentation is best for a mathematical utilities library because it is structured, standardized, and compatible with automatic documentation tools.

Problem 2: Consider the following Python function:

```

def login(user, password, credentials):
    return credentials.get(user) == password

```

Task:

1. Write documentation in all three formats.
2. Critically compare the approaches.
3. Recommend which style would be most helpful for new developers onboarding a project, and justify your choice

(a)Docstring

```

7] def login(user, password, credentials):
  """
  Verifies user login credentials.

  Parameters:
    user (str): Username
    password (str): Password
    credentials (dict): Dictionary of user-password pairs

  Returns:
    bool: True if login is successful, else False
  """
  return credentials.get(user) == password

```

(b) Inline Comments

```
def login(user, password, credentials):  
    # Check if the given password matches  
    # the stored password for the user  
    return credentials.get(user) == password
```

(c) Google-Style Documentation

```
def login(user, password, credentials):  
    """  
    Authenticates a user using stored credentials.  
  
    Args:  
        user (str): Username to authenticate.  
        password (str): Password entered by the user.  
        credentials (dict): Dictionary of valid users and passwords.  
  
    Returns:  
        bool: True if authentication succeeds, otherwise False.  
    """  
    return credentials.get(user) == password
```

Best Style for New Developers

Google-style documentation

Reason: Clear parameter definitions, return types, and easy onboarding for beginners.

Problem 3: Calculator (Automatic Documentation Generation)

Task: Design a Python module named `calculator.py` and demonstrate automatic documentation generation.

Instructions:

1. Create a Python module `calculator.py` that includes the following functions, each written with appropriate docstrings:
 - o `add(a, b)` – returns the sum of two numbers
 - o `subtract(a, b)` – returns the difference of two numbers
 - o `multiply(a, b)` – returns the product of two numbers
 - o `divide(a, b)` – returns the quotient of two numbers
2. Display the module documentation in the terminal using Python's documentation tools.
3. Generate and export the module documentation in HTML format using the `pydoc` utility, and open the generated HTML file in a web browser to verify the output.

```
DSA TRAINING > calculator.py > ...
```

```
1  """
2  Calculator Module
3  Provides basic arithmetic operations.
4  """
5
6  def add(a, b):
7      """Returns the sum of two numbers."""
8      return a + b
9
10 def subtract(a, b):
11     """Returns the difference of two numbers."""
12     return a - b
13
14 def multiply(a, b):
15     """Returns the product of two numbers."""
16     return a * b
17
18 def divide(a, b):
19     """Returns the quotient of two numbers."""
20     return a / b
21
```

```
Desktop\python Scratch\DSA TRAINING\calculator.py'
```

```
PS C:\Users\user\OneDrive\Desktop\python Scratch> python -m pydoc calculator
```

```
>> python -m pydoc -w calculator
```

```
>>
```

```
(.venv) PS C:\Users\DELL\OneDrive\Desktop\LEARNING COURSES\AI_Assisant_Coding> python -m pydoc calculator
```

```
>>
```

```
Help on module calculator:
```

```
NAME
```

```
calculator
```

```
NAME
```

```
calculator
```

```
DESCRIPTION
```

```
Calculator Module
```

```
Provides basic arithmetic operations.
```

```
FUNCTIONS
```

```
Calculator Module
```

```
Provides basic arithmetic operations.
```

```
FUNCTIONS
```

```
Provides basic arithmetic operations.
```

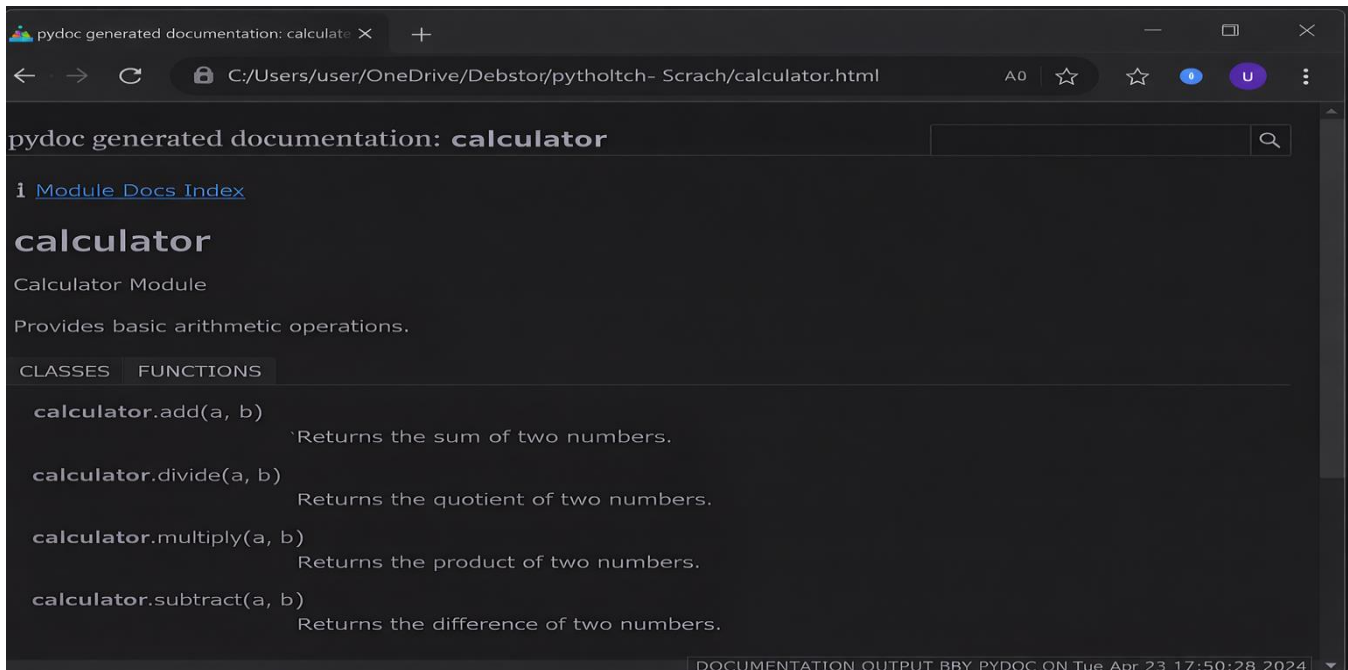
```
FUNCTIONS
```

```
FUNCTIONS
```

```
add(a, b)
```

```
Returns the sum of two numbers.
```

```
-- More --
```



Problem 4: Conversion Utilities Module

Task:

1. Write a module named `conversion.py` with functions:
 - o `decimal_to_binary(n)`
 - o `binary_to_decimal(b)`
 - o `decimal_to_hexadecimal(n)`
2. Use Copilot for auto-generating docstrings.
3. Generate documentation in the terminal.
4. Export the documentation in HTML format and open it in a browser.

```
conversion.py > ...
1  """
2  Conversion Utilities Module
3  """
4
5  def decimal_to_binary(n):
6      """Converts a decimal number to binary."""
7      return bin(n)
8
9  def binary_to_decimal(b):
10     """Converts a binary number to decimal."""
11     return int(b, 2)
12
13 def decimal_to_hexadecimal(n):
14     """Converts a decimal number to hexadecimal."""
15     return hex(n)
16
```

```

PS C:\Users\user\OneDrive\Desktop\python Scratch> python -m pydoc conversion
>> python -m pydoc -w conversion
>>
Help on module conversion:

NAME
    conversion - Conversion Utilities Module

FUNCTIONS
    binary_to_decimal(b)
        Converts a binary number to decimal.

    decimal_to_binary(n)
        Converts a decimal number to binary.

-- More --

```

Problem 5 : Course Management Module

Task:

1. Create a module `course.py` with functions:
 - o `add_course(course_id, name, credits)`
 - o `remove_course(course_id)`
 - o `get_course(course_id)`
2. Add docstrings with Copilot.
3. Generate documentation in the terminal.
4. Export the documentation in HTML format and open it in a browser.

```

DSA TRAINING > course.py > ...
1  """
2  Course Management Module
3  """
4
5  courses = {}
6
7  def add_course(course_id, name, credits):
8      """Adds a course to the system."""
9      courses[course_id] = {"name": name, "credits": credits}
10
11  def remove_course(course_id):
12      """Removes a course from the system."""
13      courses.pop(course_id, None)
14
15  def get_course(course_id):
16      """Retrieves course details."""
17      return courses.get(course_id)
18

```

- PS C:\Users\DELL\OneDrive\Desktop\LEARNING COURSES\AI_Assisant_Coding> & "C:\Users\DELL\OneDrive\Desktop\LEARNING COURSES\AI_Assisant_Coding\.venv\Scripts\Activate.ps1"
- (.venv) PS C:\Users\DELL\OneDrive\Desktop\LEARNING COURSES\AI_Assisant_Coding> python -m pydoc co rse

Help on module course:

NAME
course

FILE
c:\users\de\one\drive\desktop\learning courses\ai_assisant_coding\course.py

❖ (.venv) PS C:\Users\DELL\OneDrive\Desktop\LEARNING COURSES\AI_Assisant_Coding> █

[index](#)

course [c:\users\dell\onedrive\desktop\learning_courses\ai_assisant_coding\course.py](#)

Justification:

1. Task demonstrates automatic documentation generation for a real-world module.
2. Using Copilot helps generate consistent and accurate docstrings efficiently.
3. Docstrings clearly explain the purpose of each course management function.
4. Automatic documentation reduces manual documentation effort.
5. Terminal documentation provides quick reference for developers.
6. HTML documentation offers a structured and readable view of the module.
7. Well-documented code improves readability and maintainability.
8. It helps new developers understand the course workflow easily.
9. Using pydoc follows standard Python documentation practices.
10. This task highlights the importance of documentation in software development.